

# TP1 : Initiation à VIVADO et aux premiers designs

## Combinatoires, séquentiels et fichiers de test

TP – SN360/SN361 [2]

### Préliminaires :

Le travail demandé dans ce TP repose sur le travail à maison TM1 (qui doit être préparé et déposé sur Chamilo avant la séance de TP). **Le compte rendu du ce TP1 sera noté et devra aussi être déposé sur Chamilo [2] (voir la liste de questions à la fin de cet énoncé).**

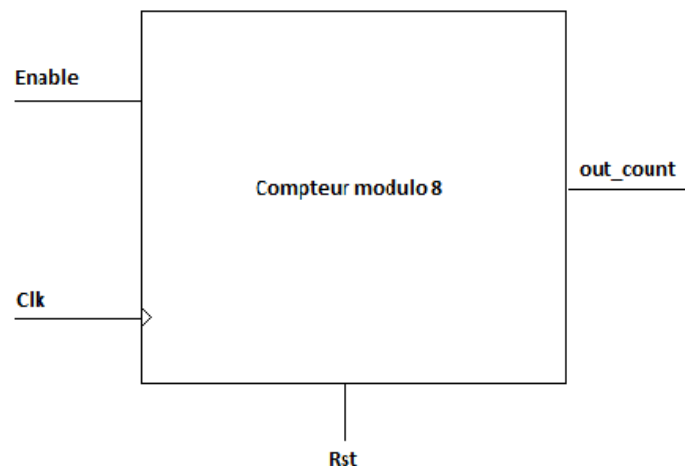
Ce qui est attendu dans votre compte rendu :

Le nom du fichier doit respecter le format suivant :  
SN36X\_TP1\_TPnumerodugroupe\_nom1\_nom2.pdf.

- Les réponses aux questions posées **en respectant la numérotation**

### Etape 1.1 : Composants à réaliser : Compteur modulo 8

La première architecture à décrire en langage VHDL est celle d'un compteur modulo qui sur 3 bits permet de compter de 0 à 7. Cette architecture sera d'abord simulée puis implémentée sur une carte à base de FPGA basys3 qui permettra de vérifier le fonctionnement du compteur sur un afficheur 7segments.



D'après la figure ci-dessus, le fonctionnement du circuit de compteur modulo 8 est comme suit :

- Si rst est mise à 1, le compteur est mis à 0.
- Si Enable est mise à 1, le compteur commence à compter alors que si Enable est mise à 0, le compteur arrête à compter.
- Alors que le clk est le signal d'horloge de notre système.

### Etape 1.2 : Réalisation du compteur modulo 8

**Analysez le fonctionnement du compteur modulo 8 fourni (compteur\_modulo8\_v1.vhd).**

Puis vérifiez son fonctionnement avec le testbench fourni (tb\_compteur\_8\_v1.vhd) en suivant les étapes ci-dessous.

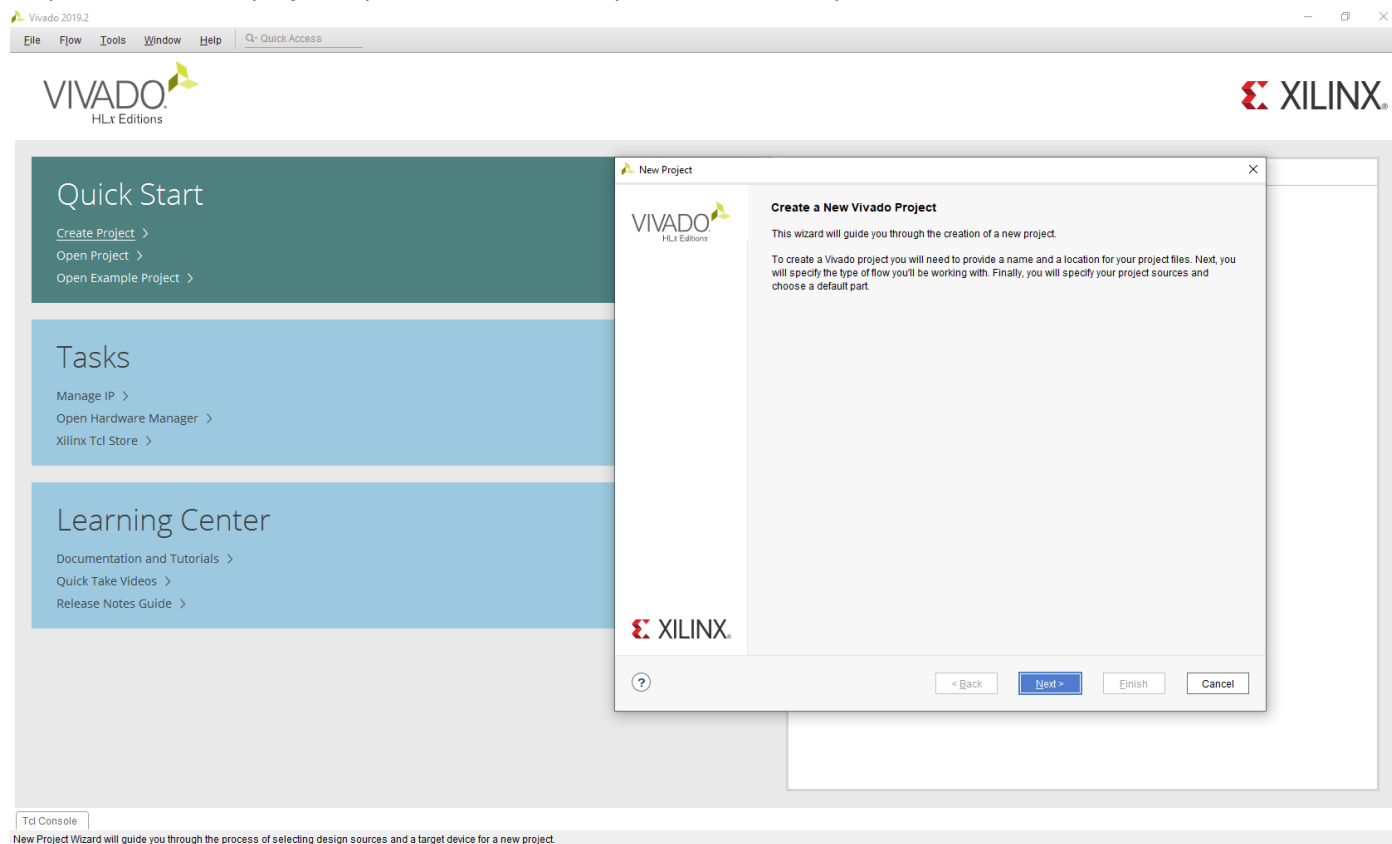
#### Etape 1.2.1 : lancement du logiciel

Lancez « VIVADO.exe », la version ici présentée est la 2019.2.

C.D.

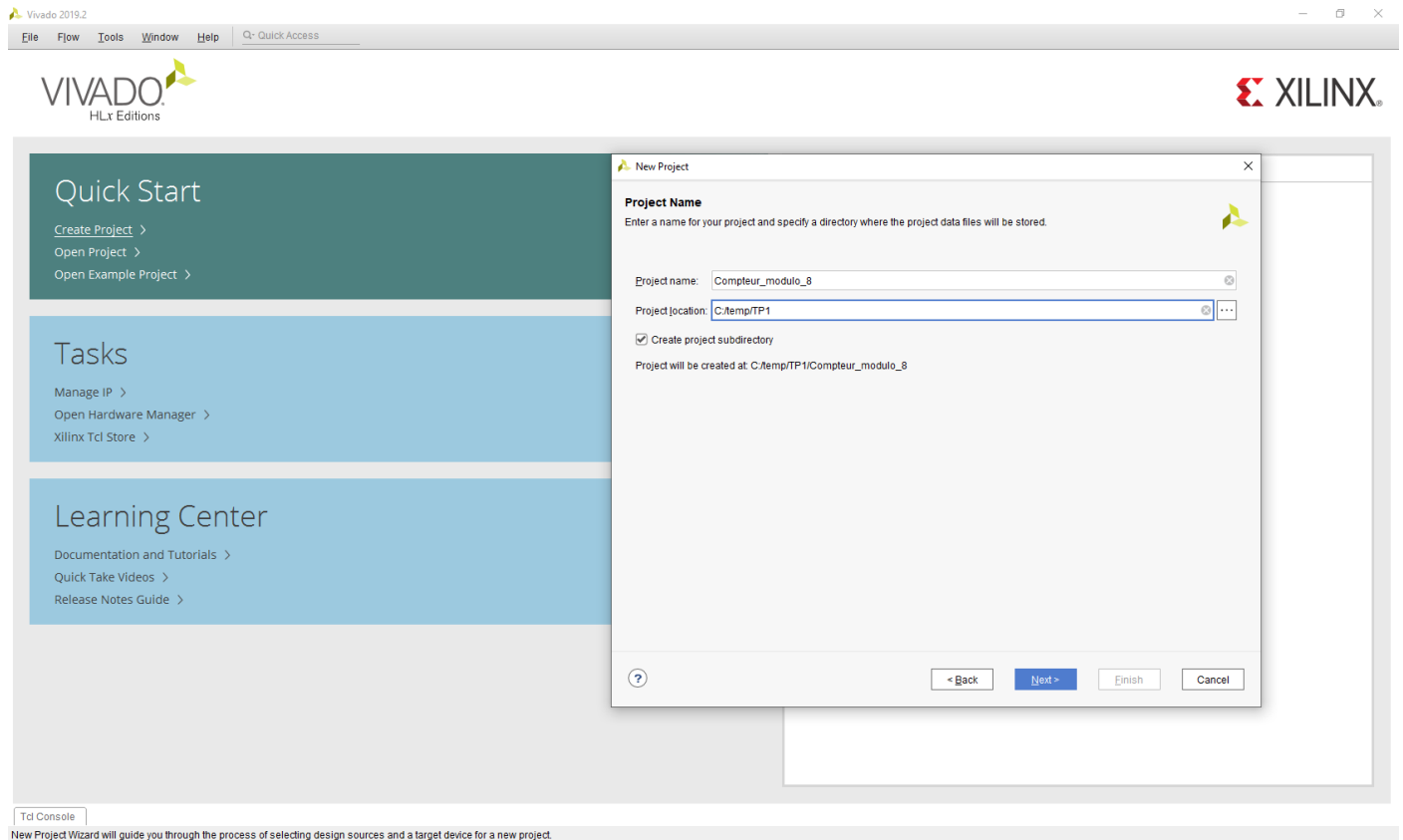
### Etape 1.2.2 création du projet

Cliquez sur « create project » puis dans la fenêtre qui est ouverte, cliquez sur “Next”.



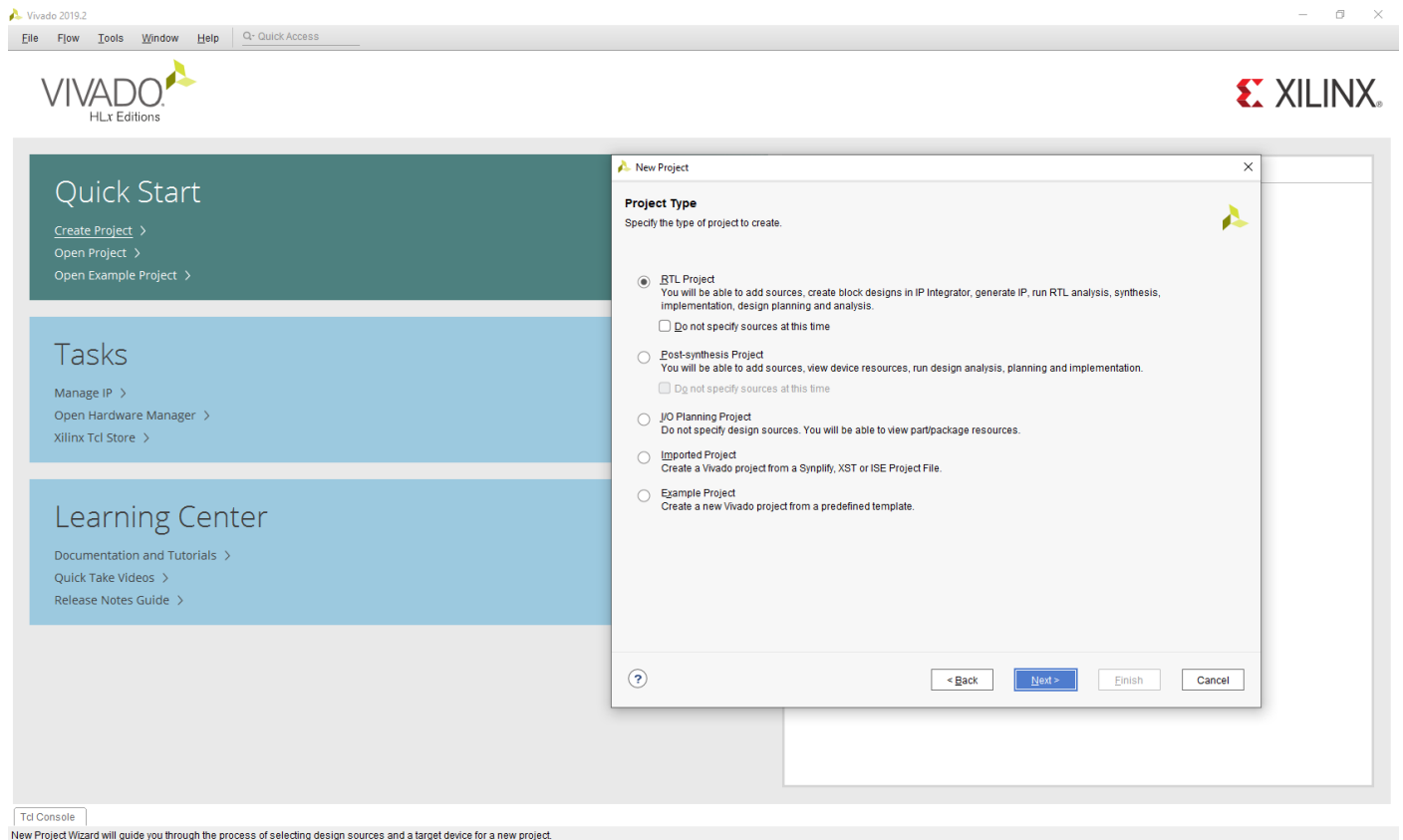
### Etape 1.2.3 : Définition du nom et de l’emplacement du projet

Renseignez les champs « Project name » et « Project location » avec le nom et l’emplacement cible de votre projet en cours de création puis cliquez sur « Next ».



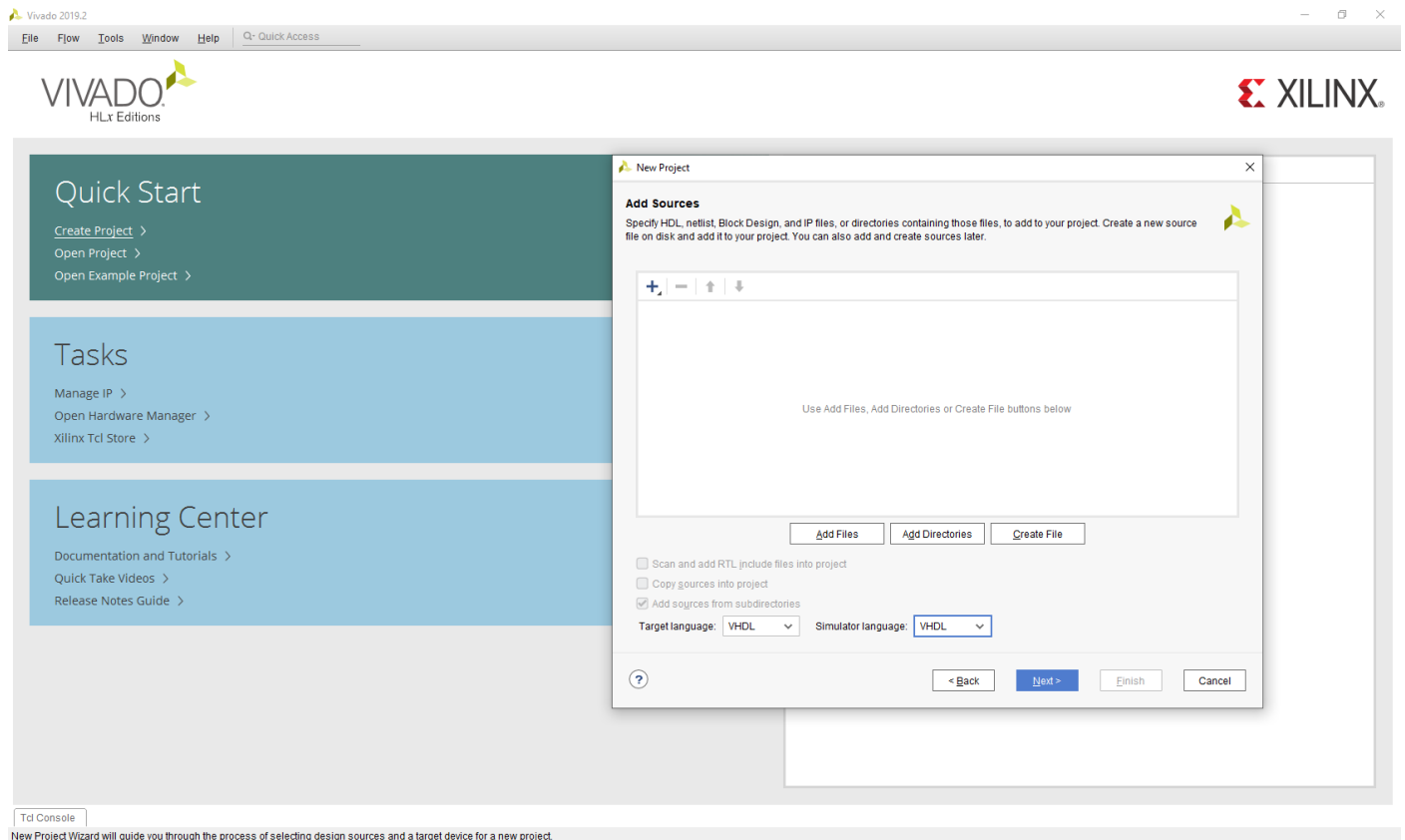
#### Etape 1.2.4 : Définition du type de projet

Cochez la case comme ci-dessous et cliquez sur « Next ».



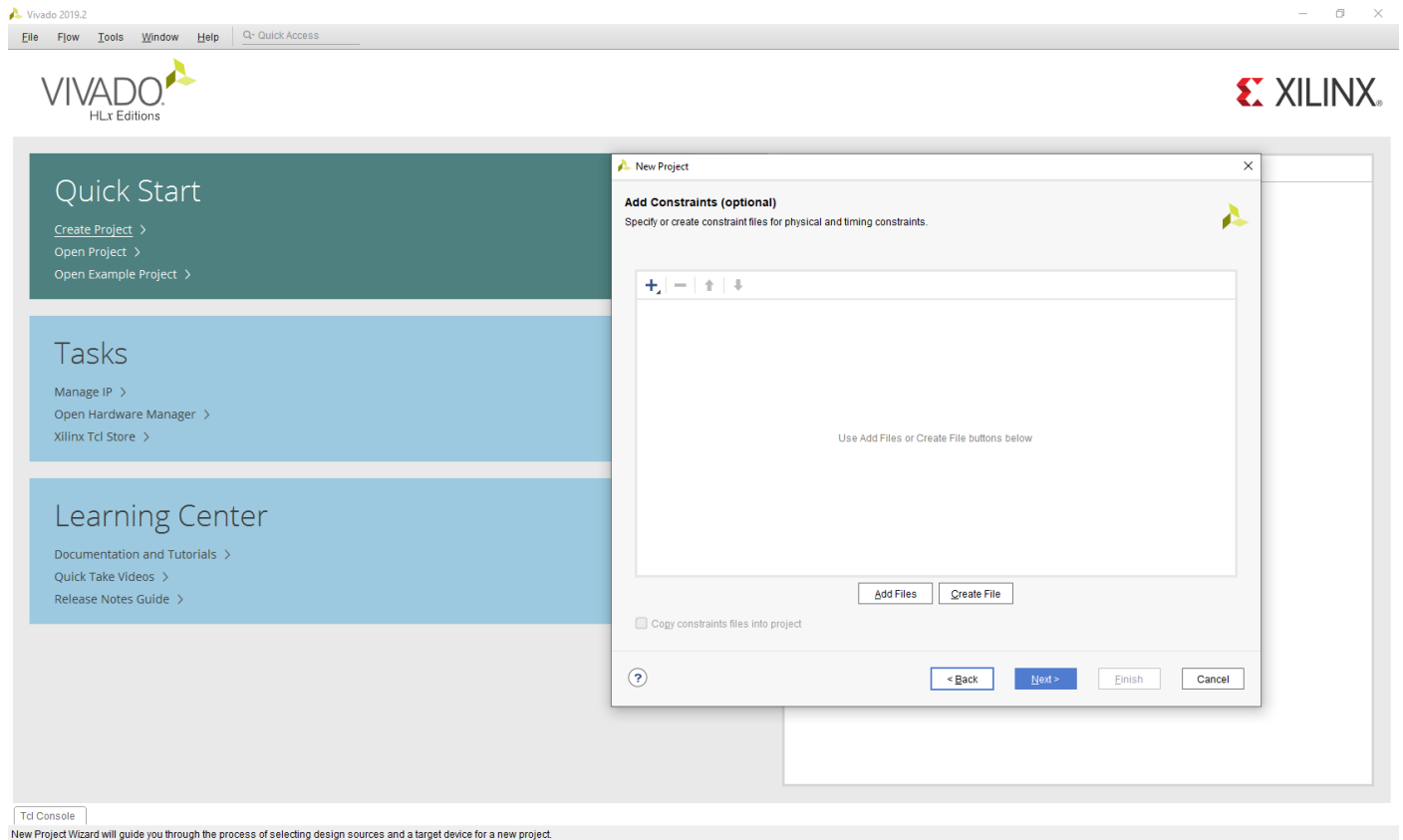
#### Etape 1.2.5 : ajout de fichiers déjà rédigés et sélection du langage utilisé

Lors de cette étape vous devez ajouter les fichiers déjà rédigés si cela est le cas (dans notre cas, aucun fichier n'est à ajouter). Renseignez dans le « Target language » et le « Simulator language » l'option VHDL. (Dans ce TP seul le VHDL sera utilisé). Ensuite, cliquez sur « Next ».



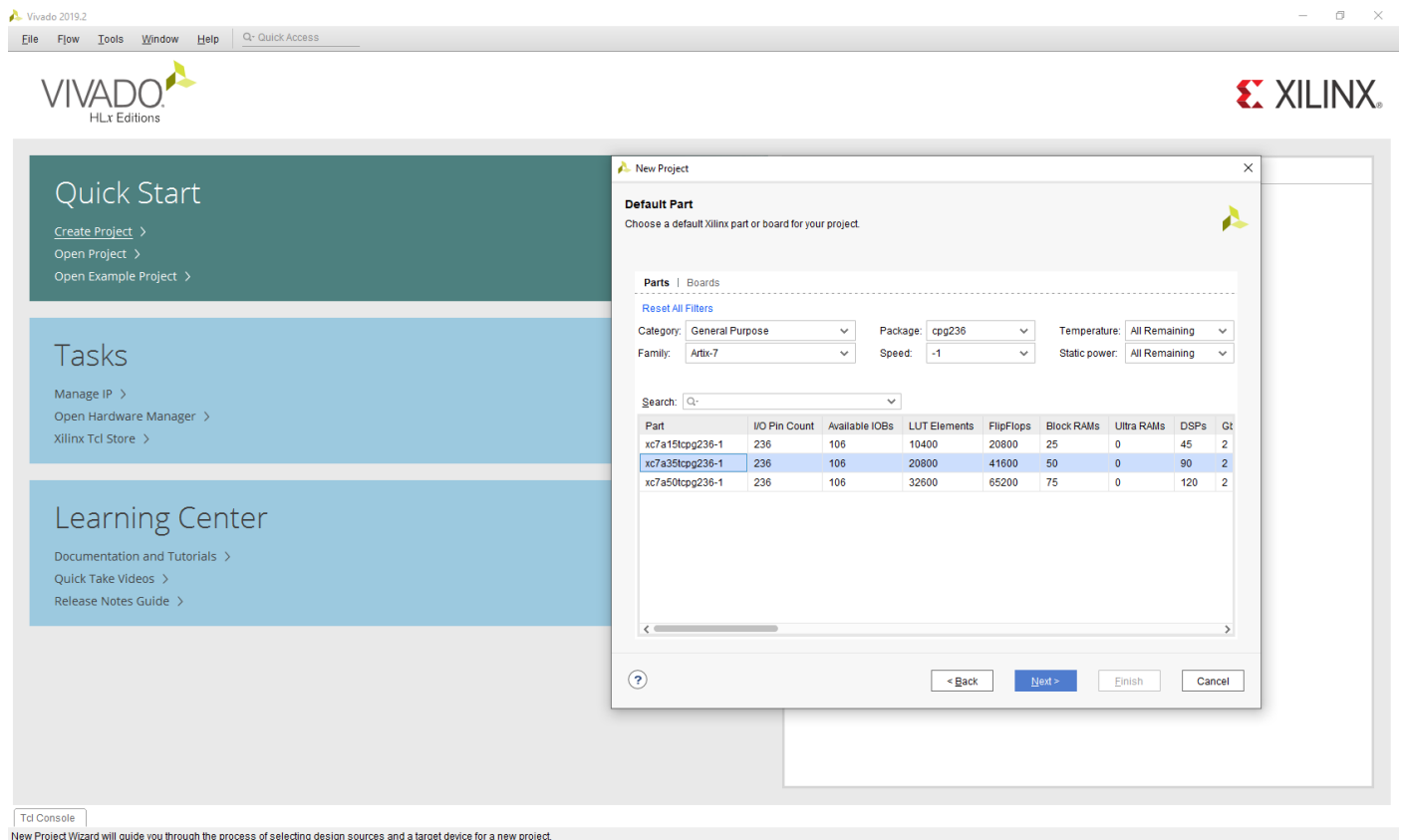
### Etape 1.2.6 : Ajout d'IPs/Constraints

Lors de cette étape cliquez sur « Next ». (Cette étape est trop avancée pour vous à ce stade).  
Effectuez du même pour l'étape suivante.



### Etape 1.2.7 : Sélection du composant FPGA cible

Choisissez la carte convenable pour notre projet, l'Artix7 XC7a35tcbg236-1 et puis cliquez sur « Next ».

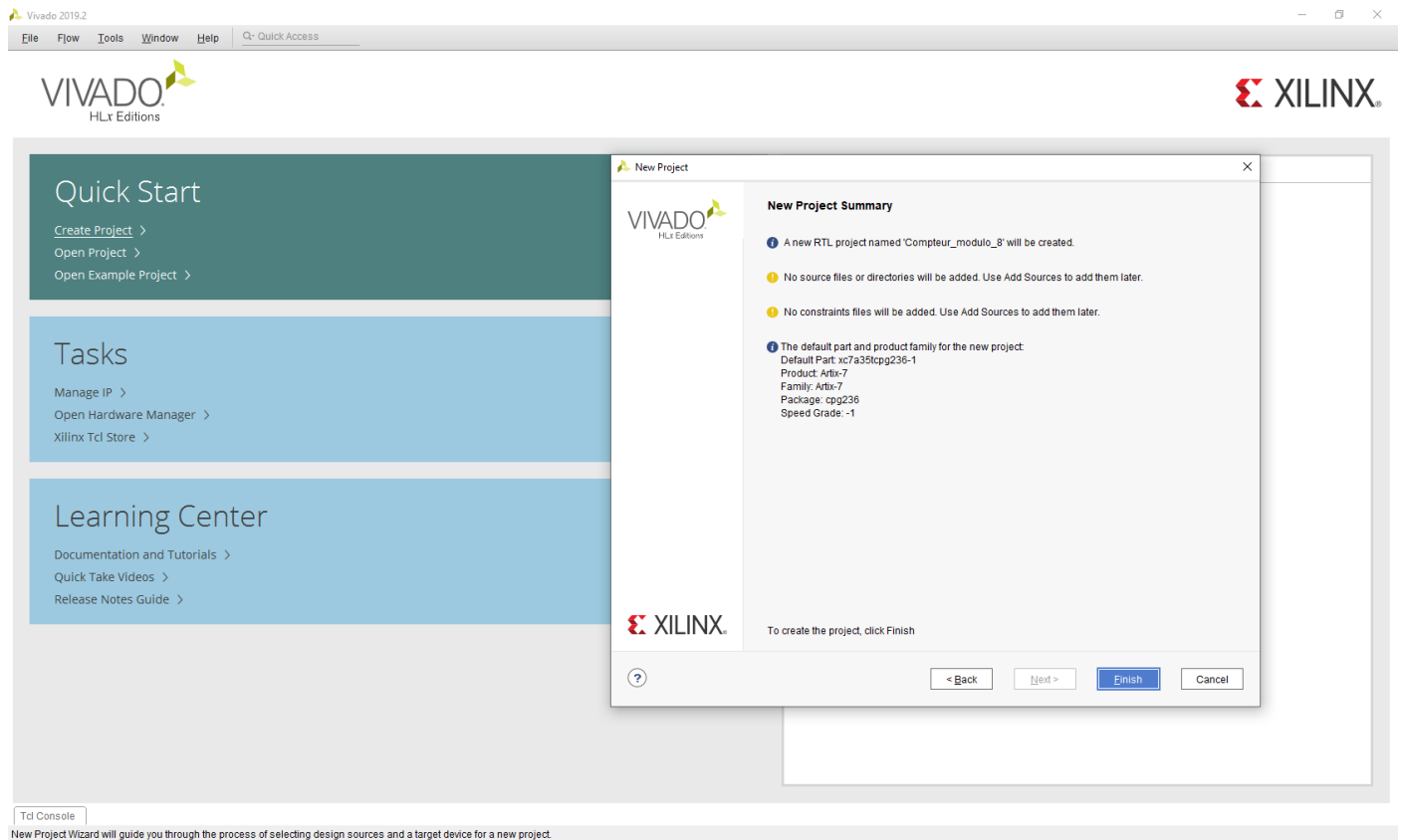


### Etape 1.2.8 : Validation du projet

Cliquez sur « Finish » si vous avez le même récapitulatif présenté ci-dessous :

C.D.

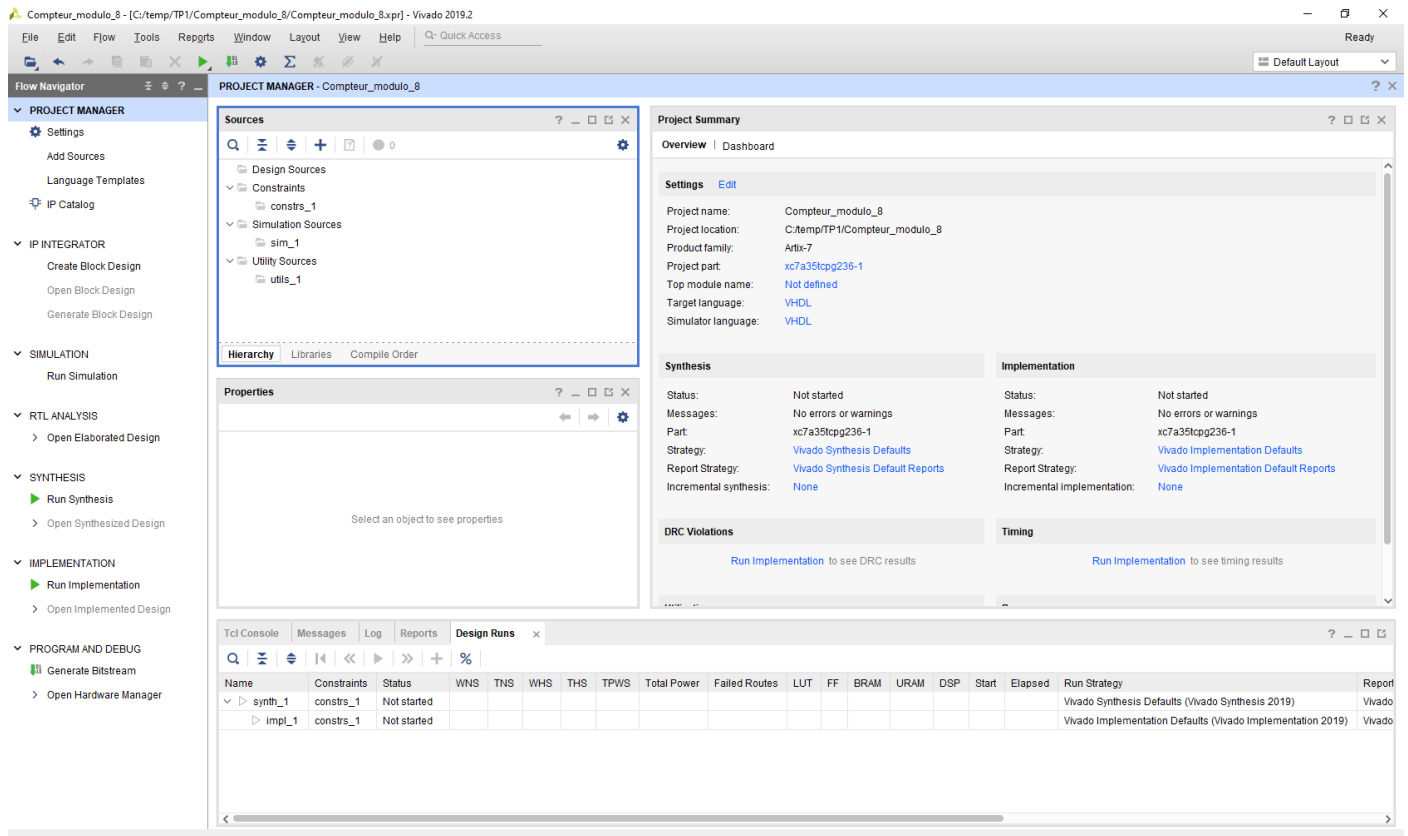
- Pas de sources
- Pas d'IPs
- Pas de contraintes



### Etape 1.2.9 : Présentation de l'interface projet de VIVADO

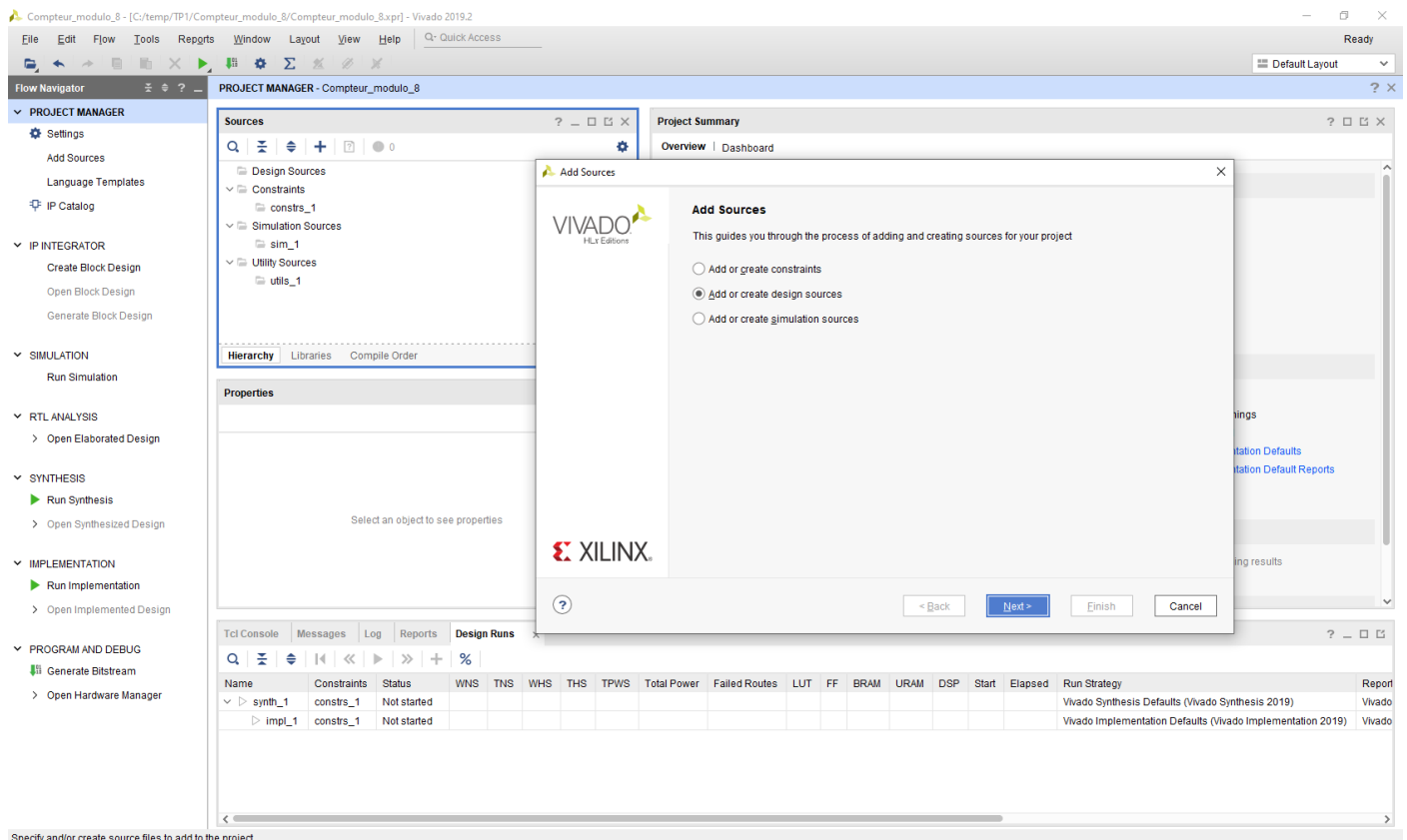
Ici plusieurs fenêtres sont importantes :

- « Flow Navigator » vous propose des certains nombres d'actions sur le flow de conception/implémentation du projet
- « Project Manager » affiche la structure/hierarchie des fichiers présents dans le projet
- « Project Summary » affiche les informations relatives aux paramètres projet et les résultats de synthèse et implémentation.

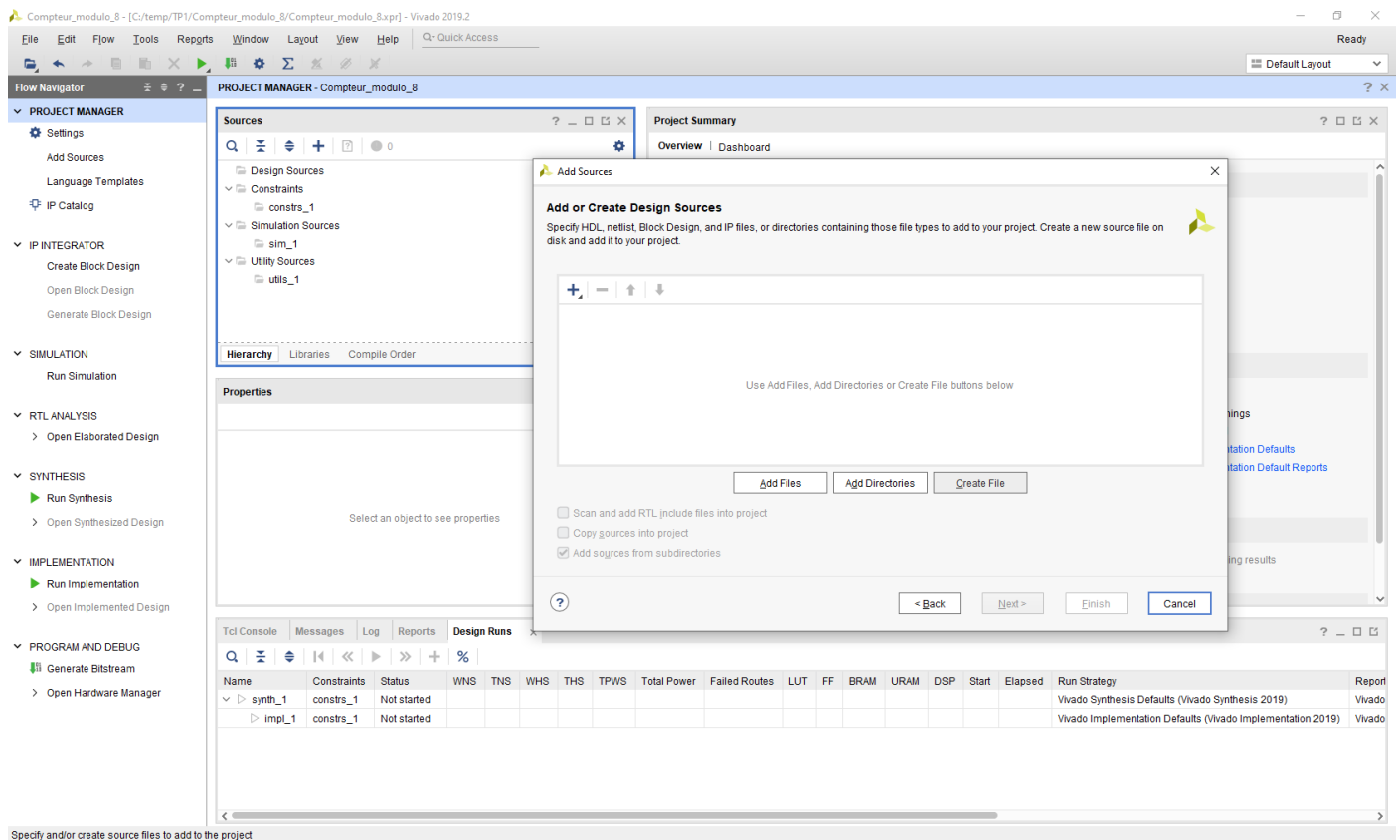


### Etape 1.2.10 : Création du premier fichier

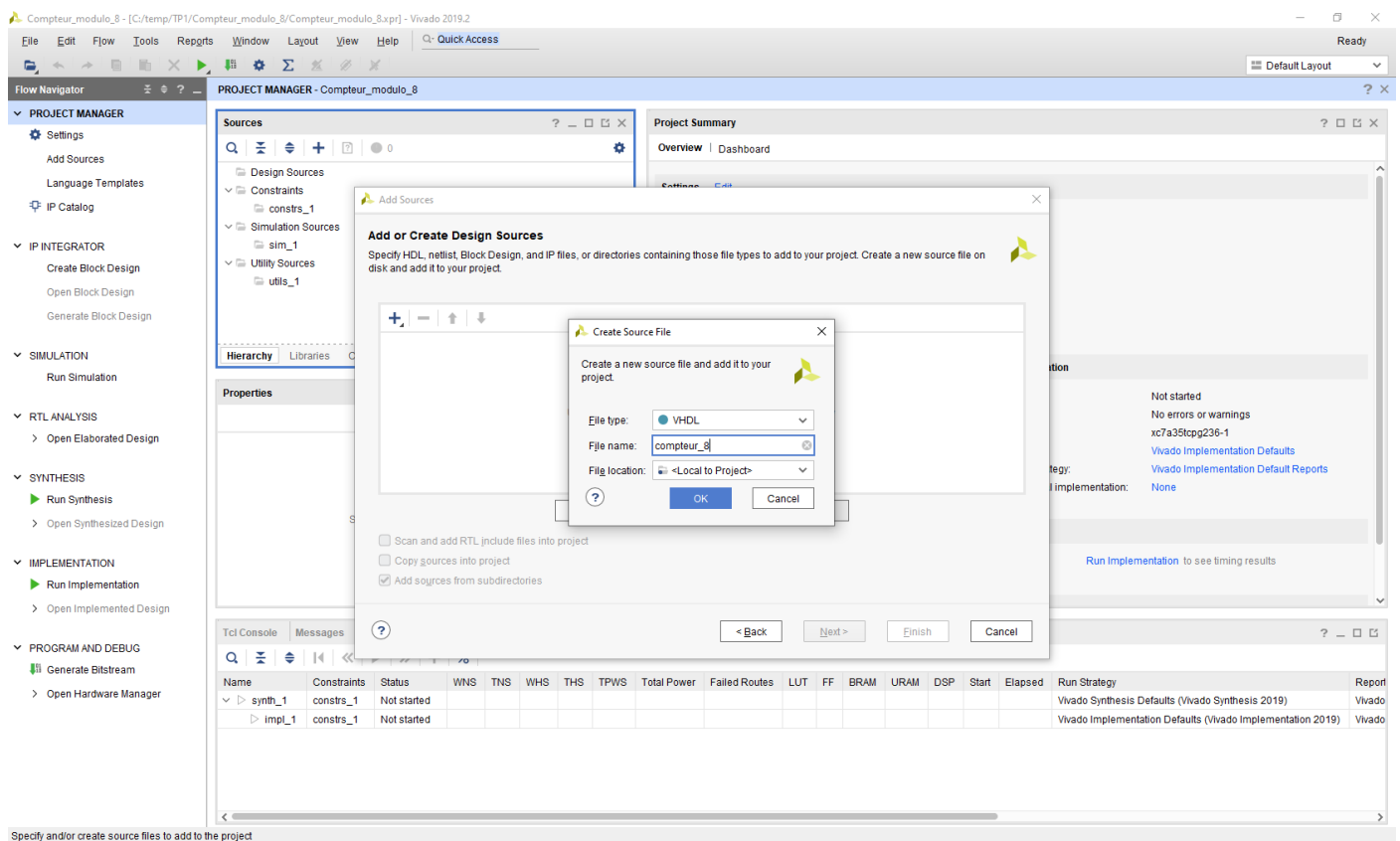
Cliquez sur « Add Sources » puis « Add or create design sources » puis « Add Files » :



Cliquez sur « Create File » :

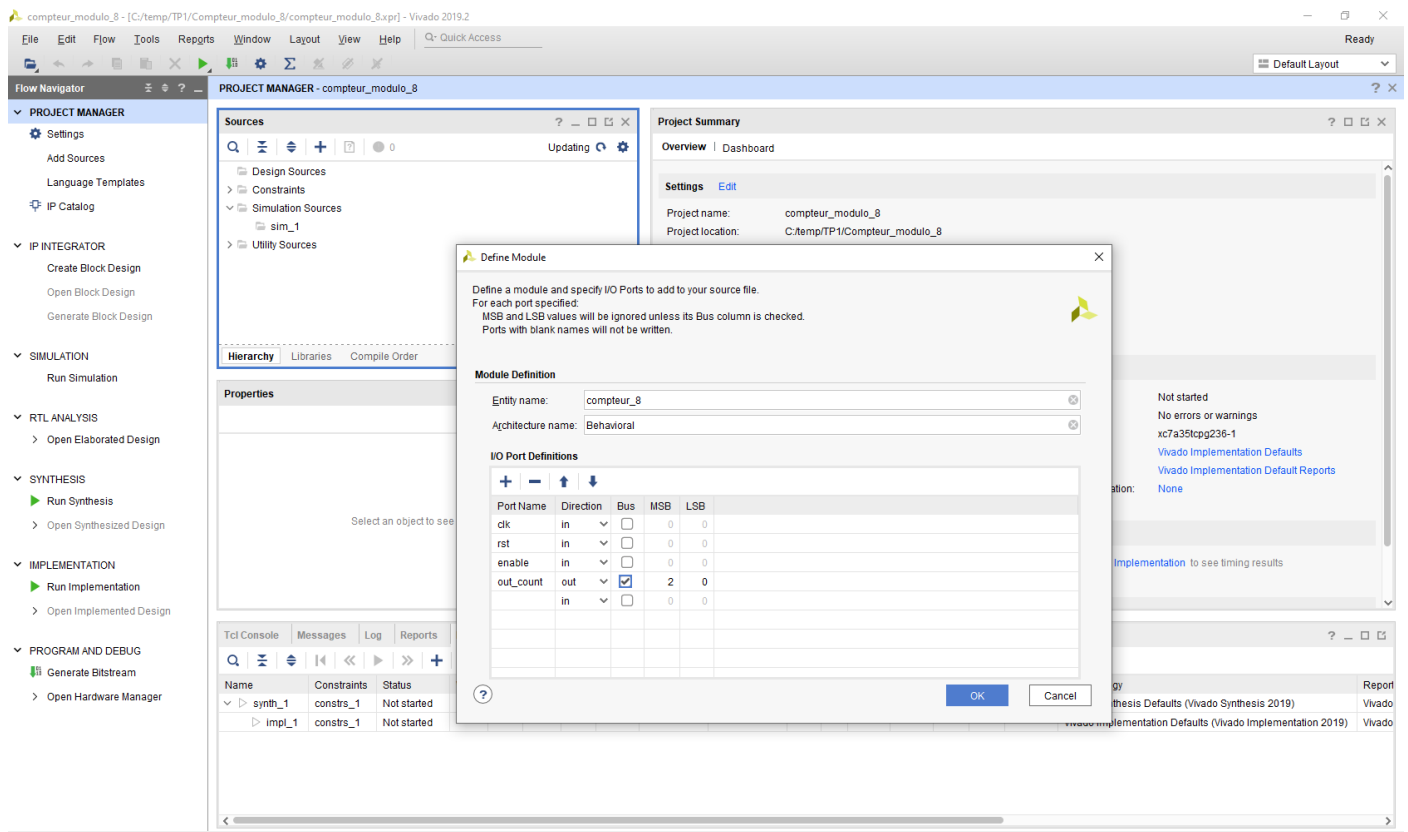


Remplir les cases comme suit puis cliquez sur « OK » et « Finish » :

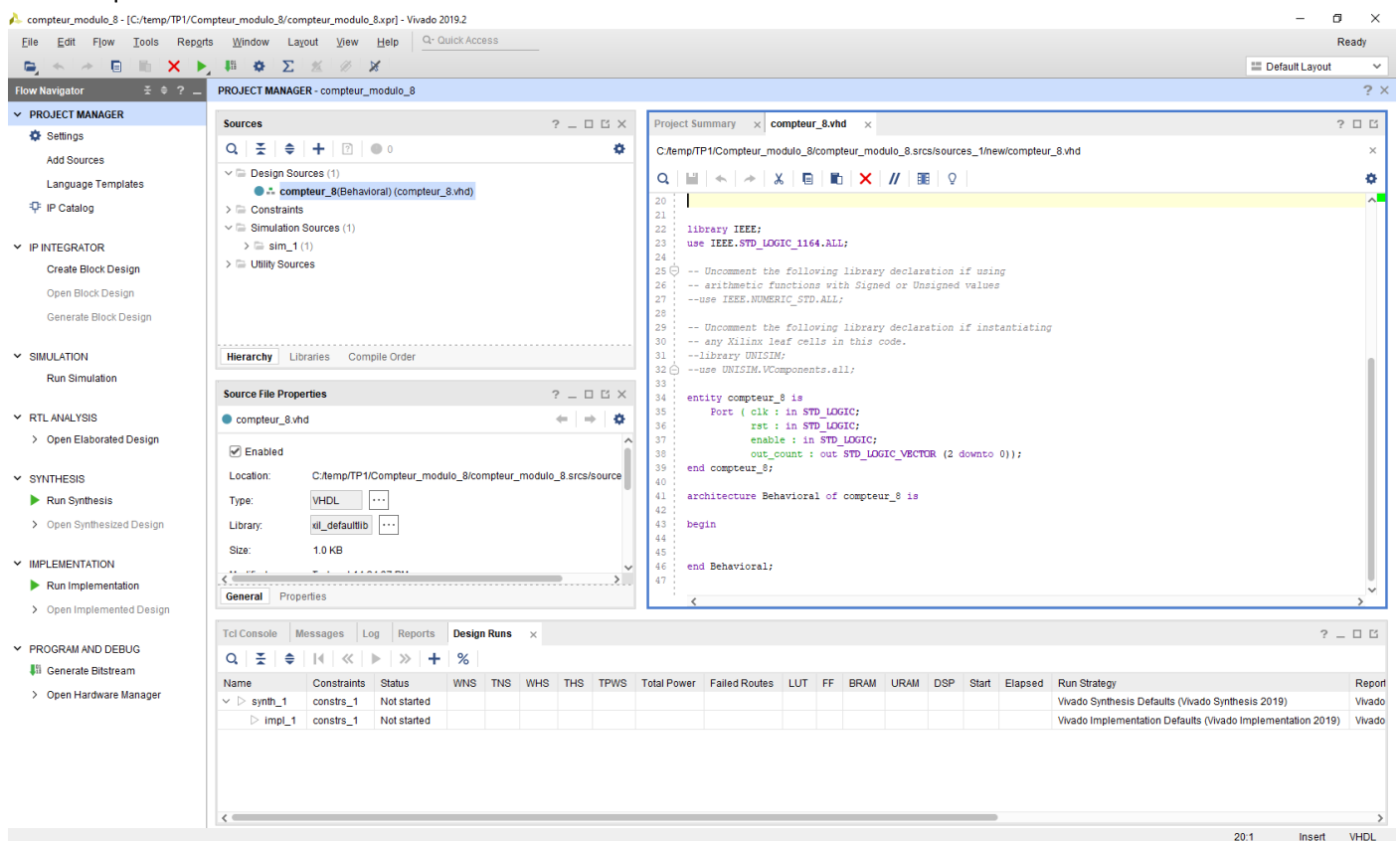


Remplir les cases par les I/O correspondants à l'architecture du compteur modulo 8 ainsi que le type de chaque entrée et sortie :



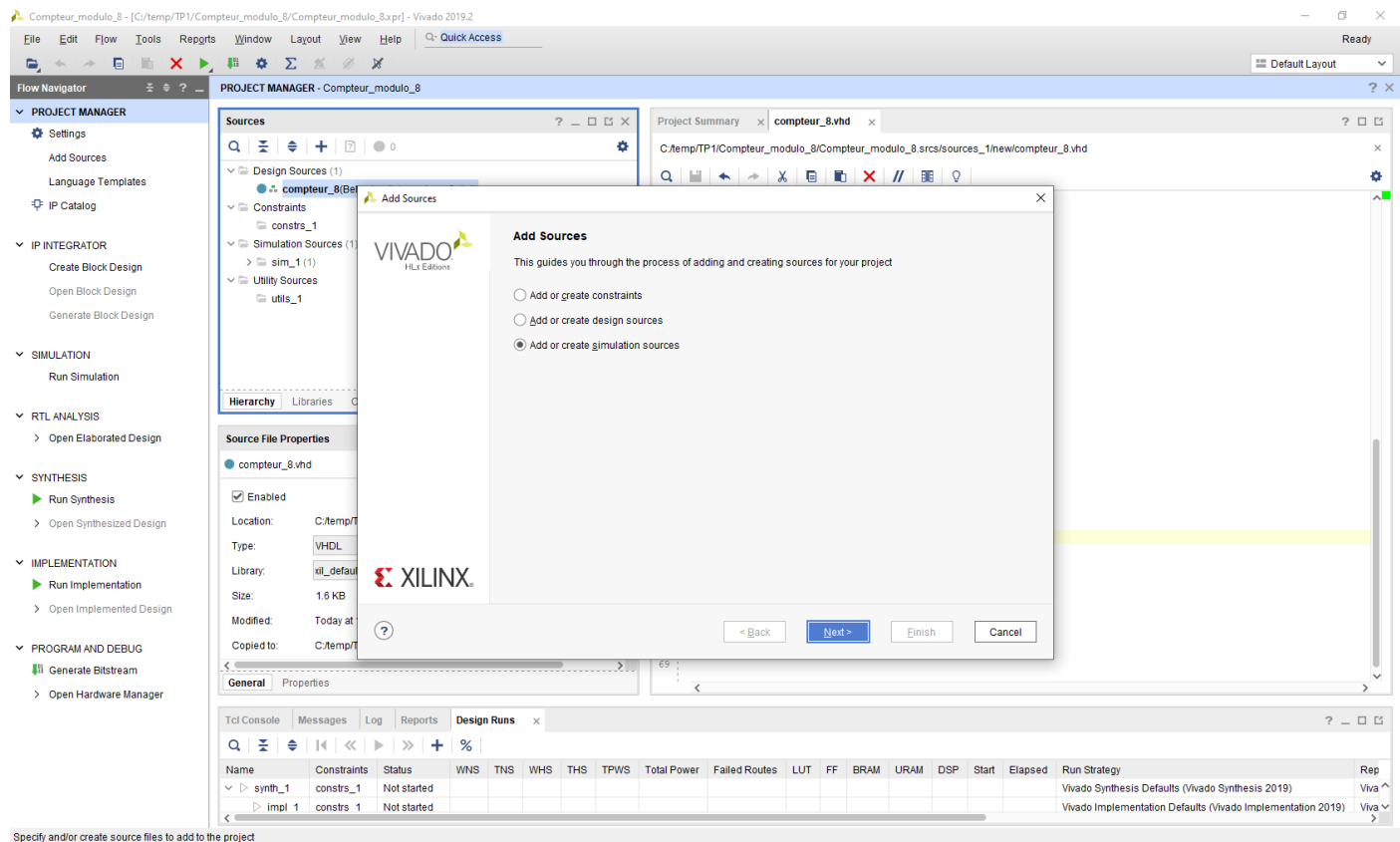


Vérifiez que vous avez obtenu la même structure du code si dessous :

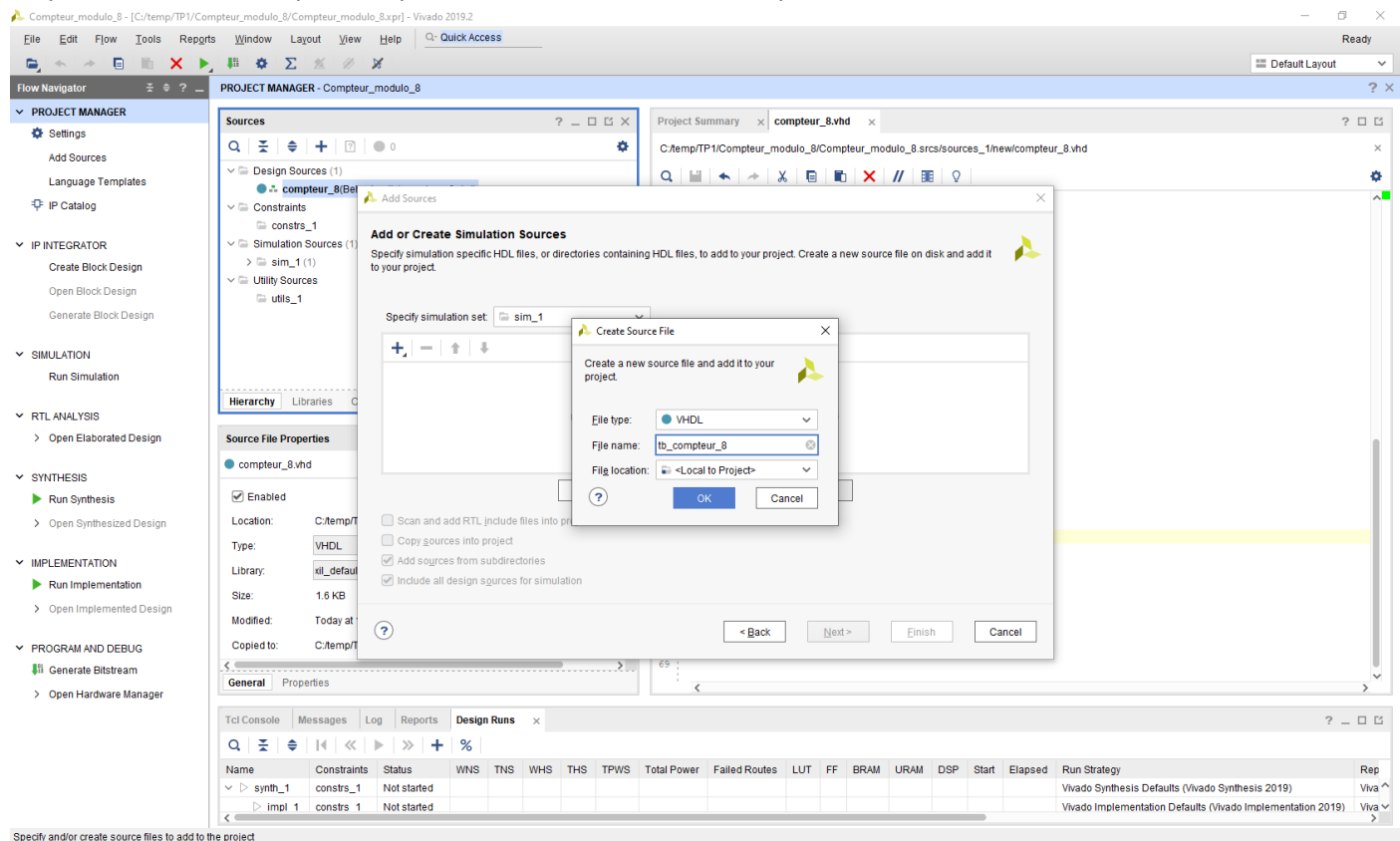


### Etape 1.2.11 : Ajout de fichier de simulation/test

Cliquez sur « Add Sources » puis « Add or create simulation sources » puis « Add Files » :

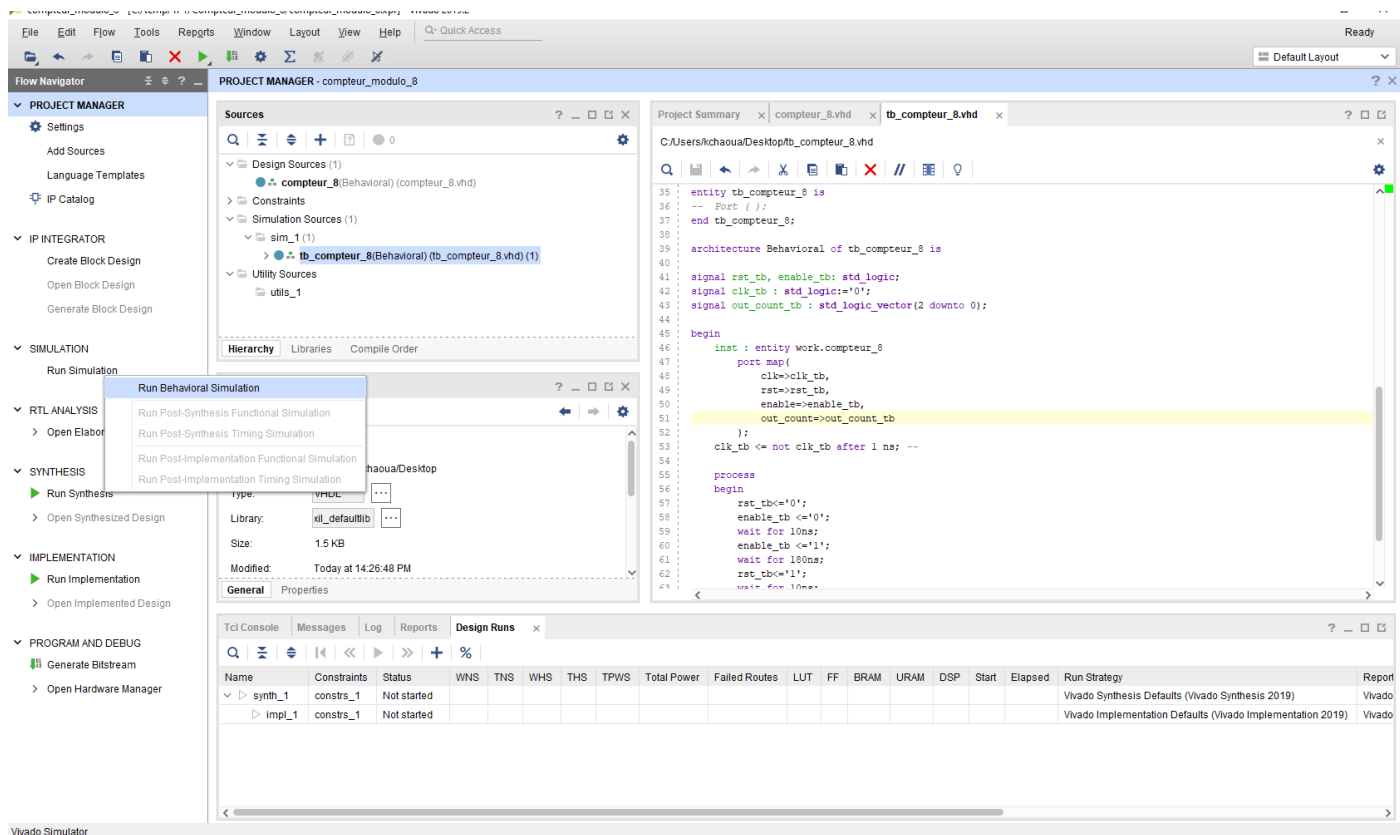


Cliquer sur « Add Files» puis remplir les cases avec le nom correspondant à votre tetbench :

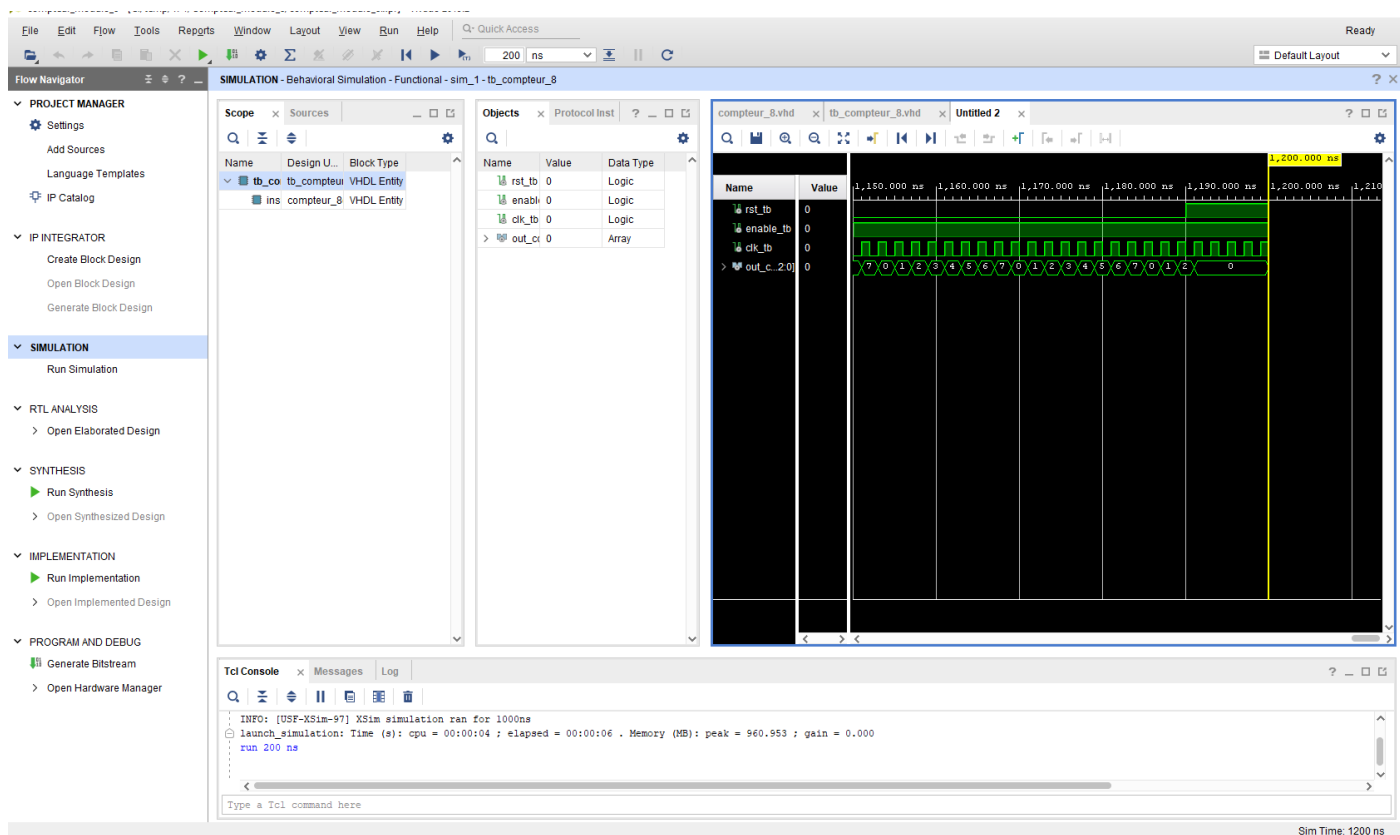


**Etape 1.2.12 : Lancer la simulation**

Dans le « Flow Navigator » cliquez sur « Simulation».  
Vérifiez que le fichier à simuler est bien l'entité « tb\_compteur\_8\_v1 ».



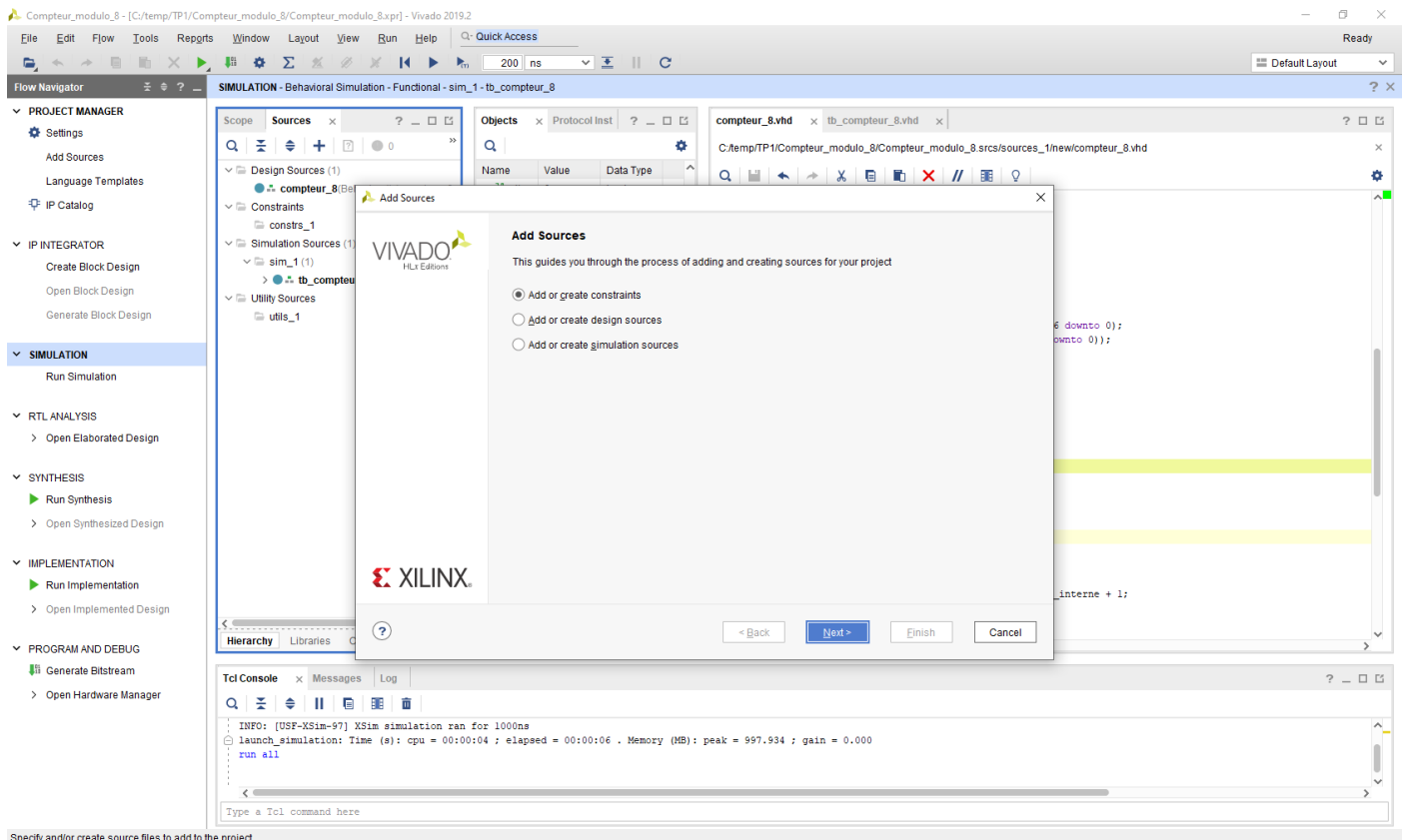
Choisir Run for 200ns au début pour mieux comprendre.

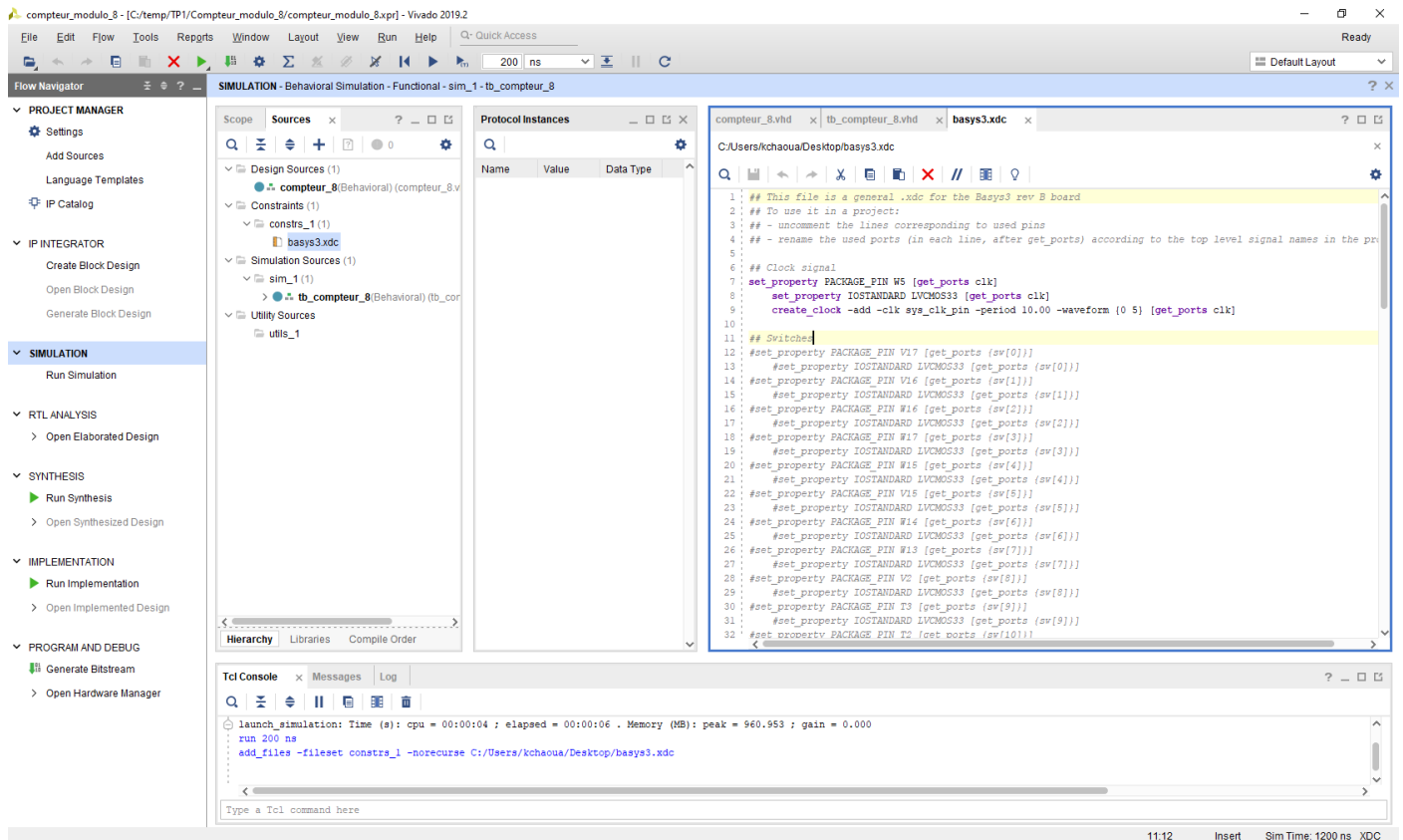


### Etape 1.3 : Implémentation d'un compteur modulo 8 sur FPGA

On modifie la description du compteur modulo 8 par l'ajout de sortie vers l'afficheur 7 segments et de process supplémentaires. (Téléchargez le nouveau fichier compteur\_8\_v2.vhd).

Afin d'implémenter compteur modulo 8, on doit ajouter un fichier basys3.xdc (Téléchargez-le). Suivre les conseils de **l'enseignant** afin de créer le fichier bitstream qui va être implémenté sur la carte Basys3 à base de FPGA.





Modifiez le dernier fichier ajouté en nommant les ports illustrés dans le xdc file par la même nomination du Top Level compteur\_8.vhd. Comme par exemple le signal d'horloge est nommé "clk" dans le Top level et il est connecté au pin "W5" de la FPGA :

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -clk sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## LEDs
set_property PACKAGE_PIN U16 [get_ports out_count[0]]
set_property IOSTANDARD LVCMOS33 [get_ports out_count[0]]
set_property PACKAGE_PIN E19 [get_ports out_count[1]]
set_property IOSTANDARD LVCMOS33 [get_ports out_count[1]]
set_property PACKAGE_PIN U19 [get_ports out_count[2]]
set_property IOSTANDARD LVCMOS33 [get_ports out_count[2]]
set_property PACKAGE_PIN V19 [get_ports {deb}]
set_property IOSTANDARD LVCMOS33 [get_ports {deb}]

##7 segment display
set_property PACKAGE_PIN W7 [get_ports {seven_seg_out[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[0]}]
set_property PACKAGE_PIN W6 [get_ports {seven_seg_out[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[1]}]
set_property PACKAGE_PIN U8 [get_ports {seven_seg_out[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[2]}]
set_property PACKAGE_PIN V8 [get_ports {seven_seg_out[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[3]}]
set_property PACKAGE_PIN U5 [get_ports {seven_seg_out[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[4]}]
set_property PACKAGE_PIN V5 [get_ports {seven_seg_out[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[5]}]
set_property PACKAGE_PIN U7 [get_ports {seven_seg_out[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seven_seg_out[6]}]
set_property PACKAGE_PIN V7 [get_ports dp]
set_property IOSTANDARD LVCMOS33 [get_ports dp]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

|

##Buttons
set_property PACKAGE_PIN U18 [get_ports enable]
set_property IOSTANDARD LVCMOS33 [get_ports enable]
set_property PACKAGE_PIN T18 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
```

NB : Dans le but de créer le fichier bitstream à implémenter sur la carte FPGA, il faut passer par les étapes du flot de conception et de vérification. Donc à chaque étape, vous pouvez consulter les erreurs (si c'est le cas) dans le TCL Console en bas de la fenêtre.

**Questions :**

1. Quel est le rôle du fichier compteur.vhd?
2. A quoi sert l'utilisation d'un fichier testbench?
3. Quel est le résultat de la synthèse? dans votre cas combien de flip flop sont générées?
4. **Quelles sont les étapes du flot de conception et de vérification?**
5. Quel est le rôle du fichier basys3.xdc? Explique le fonctionnement du bouton btn correspondante au Pin "U18"?
6. Expliquez le fonctionnement de l'afficheur 7 segments et le fonctionnement de la dernière description.

**Simulation du compteur modulo 16.**

**En simulant le compteur modulo 16 demandé dans la préparation TM1, vérifiez que les chronogrammes obtenus (à faire valider par l'enseignant pendant la séance) sont bien identiques à ceux réalisés manuellement.**

Sauvegarder votre projet sur clé USB ou sur les répertoires partagés de l'école (tp) pour retrouver vos fichiers lors de la prochaine séance.

Maintenant vous êtes prêts pour les TP suivant !